Vaardigheden – bewijs uit QuizApp.ZonderDatabase

15 vaardigheden uit het Vaardighedenportfolio Software Developer · console-app zonder database · met code, wat ik zeg en vragen van de docent (CGI)

#	Onderdeel – nr.	Ik kan…	Bewijs (waar)
1	1. Software ontwerpen – 1	vanuit een probleem bepalen welke functionaliteiten nodig zijn.	QuizApp.ZonderDatabase → Program.cs → Main() (menu) (regel 7–44)
2	1. Software ontwerpen – 5	bepalen welke gegevens mijn applicatie nodig heeft.	QuizApp.ZonderDatabase → Question.cs + QuizResult.cs (klassen Question, QuizResult, QuizAnswer)
3	1. Software ontwerpen – 9	een one-to-many-relatie uitleggen en toepassen.	QuizApp.ZonderDatabase → QuizResult.cs (List<QuizAnswer> Answers) + ResultsScreen.cs → ShowAnswers() (regel 44–57)
4	3. UX & frontend – 1	een scherm maken dat voor een gebruiker begrijpelijk is.	QuizApp.ZonderDatabase → QuizScreen.cs → ShowQuestion() (regel 42–52) + Start()
5	3. UX & frontend – 5	schermen voorzien van duidelijke labels.	QuizApp.ZonderDatabase → QuestionsScreen.cs → ReadText() + ReadQuestion() (regel 64–91)
6	3. UX & frontend – 2	bepalen wanneer een gebruiker feedback nodig heeft.	QuizApp.ZonderDatabase → QuestionsScreen.cs → AddQuestion() + QuizScreen.cs → FinishQuiz()
7	3. UX & frontend – 3	duidelijke fout- en succesmeldingen maken.	QuizApp.ZonderDatabase → QuestionsScreen.cs → DeleteQuestion() + CheckInput() (meldingen)
8	3. UX & frontend – 4	een logische navigatiestructuur ontwerpen.	QuizApp.ZonderDatabase → Program.cs → Main() + QuestionsScreen.cs → Start() (menu's met while)
9	5. Programmeren & codekwaliteit – 1	bestaande code lezen en uitleggen wat deze doet.	QuizApp.ZonderDatabase → QuizScreen.cs → FinishQuiz() (regel 89–111)
10	5. Programmeren & codekwaliteit – 2	code opdelen in logische functies/methodes.	QuizApp.ZonderDatabase → QuizScreen.cs → Start() gebruikt ShowQuestion(), GetGivenAnswer(), CheckAnswer() en FinishQuiz()
11	5. Programmeren & codekwaliteit – 4	herhaalde code refactoren.	QuizApp.ZonderDatabase → QuestionsScreen.cs → ReadText() en AskQuestionNumber()
12	5. Programmeren & codekwaliteit – 5	duidelijke namen kiezen voor variabelen, methodes en classes.	QuizApp.ZonderDatabase → QuizScreen.cs → methodenamen ShowQuestion, GetGivenAnswer, CheckAnswer, FinishQuiz + variabelen
13	7. Testen & kwaliteit – 2	happy flows én foutscenario's bedenken.	QuizApp.ZonderDatabase → QuizScreen.cs → Start() (regel 8–22) + testtabel
14	7. Testen & kwaliteit – 4	controleren of gebruikersinvoer correct wordt afgehandeld.	QuizApp.ZonderDatabase → QuestionsScreen.cs → CheckInput() (regel 93–114) + AskQuestionNumber()
15	7. Testen & kwaliteit – 5	edge cases bedenken.	QuizApp.ZonderDatabase → QuizScreen.cs → GetGivenAnswer() + CheckAnswer() (regel 54–87)

> **Tips voor het gesprek (CGI):**
• Lees eerst de pagina **"Hoe werkt de app?"** en doe de 3 oefeningen. Dan begrijp je de code echt.
• Zet de app en Visual Studio open klaar. Laat eerst zien: student maakt de quiz → docent bekijkt het resultaat.
• Leg in je eigen woorden uit. Leer de tekst niet uit je hoofd: de docent stelt vervolgvragen.
• Weet je iets niet? Zeg dat eerlijk en vertel hoe je het zou uitzoeken.
• De regelnummers in dit document zijn dezelfde als in Visual Studio.

LEES DIT EERST

Hoe werkt de app? (console, zonder database)

Dit is een **console-app**: er zijn geen formulieren, knoppen of pop-ups. De app schrijft tekst met `Console.WriteLine` en leest wat de gebruiker typt met `Console.ReadLine`. Er is **geen database**: alles staat in **lijsten** in het geheugen.

De bestanden – lees ze in deze volgorde

#	Bestand	Wat staat erin?
1	Question.cs	Hoe één vraag eruitziet (tekst, A, B, C, juiste antwoord).
2	QuizResult.cs	Hoe één poging en één antwoord eruitzien.
3	QuizData.cs	De 2 lijsten (vragen en resultaten) + de vaste vragen (hard-coded).
4	Program.cs	Start: eerst de vragen in de lijst zetten, dan het menu tonen.
5	QuizScreen.cs	Student: naam → vragen → score.
6	QuestionsScreen.cs	Docent: vragen tonen, toevoegen, wijzigen, verwijderen.
7	ResultsScreen.cs	Docent: alle pogingen en de antwoorden.

Het belangrijkste bestand: QuizData.cs

C# QuizData.cs — QuizApp.ZonderDatabase › QuizData.cs › regel 5–14

```
5     public static class QuizData
6     {
7         public static List<Question> Questions = new List<Question>();
8         public static List<QuizResult> Results = new List<QuizResult>();
9
10        public static void AddStartQuestions()
11        {
12            Questions.Add(new Question("Wat is een programmeertaal?", "Cobra", "Python", "Mamba", "B"));
13            Questions.Add(new Question("Waar staan de beste stuurlui volgens het spreekwoord?", "Aan land", "Aan het roer", "Aan wal", "C"));
14            Questions.Add(new Question("Hoe vaak stoot een ezel zich over het algemeen aan een steen?", "Nooit", "Meer dan 1 keer", "Niet 2 keer", "C"));
```

QuizData.cs (regel 5–14)

- `List<Question> Questions` = een lijst met vragen. Zoals een rij bakjes: bakje 0, bakje 1, bakje 2 ...
- `static` = er is maar één lijst voor de hele app. Elk scherm schrijft `QuizData.Questions` en krijgt dezelfde lijst.
- **Hard-coded** = de vragen staan vast in de code (in totaal 37 vragen, waarvan 2 open vragen).
- **Let op:** sluit je de app, dan zijn nieuwe vragen en resultaten weg. Het geheugen wordt leeg gemaakt.

Console-opdrachten die je moet kennen

Code	Wat doet het?	Waar?
`Console.WriteLine(x)`	tekst tonen en daarna naar een nieuwe regel	overal
`Console.Write(x)`	tekst tonen, op dezelfde regel blijven (de gebruiker typt erachter)	ReadText, menu "Kies: "
`Console.ReadLine()`	wachten tot de gebruiker iets typt en op Enter drukt; geeft een string	Start, GetGivenAnswer, ReadText
`.Trim()`	spaties voor en achter weghalen	GetGivenAnswer, ReadText
`int.TryParse(s, out n)`	probeer tekst om te zetten naar een getal; `false` als dat niet lukt	AskQuestionNumber, ResultsScreen

Lijst-opdrachten die je moet kennen

Code	Wat doet het?	Waar?
`lijst.Add(x)`	x achteraan toevoegen	AddQuestion, FinishQuiz
`lijst[i]`	het item op plek i pakken	QuizScreen.Start, ResultsScreen.Start
`lijst[i] = x`	het item op plek i vervangen	EditQuestion
`lijst.RemoveAt(i)`	het item op plek i weghalen	DeleteQuestion
`lijst.Count`	hoeveel items er zijn	QuizScreen.Start, AskQuestionNumber
`foreach (... in lijst)`	langs alle items gaan	FinishQuiz, ShowAnswers

Oefening om het echt te leren: 1) Voeg in QuizData.cs zelf een vraag toe en start de app (F5). 2) Zet een breakpoint (F9) in `CheckAnswer` en kijk met F10 wat `given` en `correct` zijn. 3) Verander in `FinishQuiz` de tekst van de score naar je eigen tekst.

1 Ik kan vanuit een probleem bepalen welke functionaliteiten nodig zijn.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → Program.cs → Main() (menu) (regel 7–44)

C# Program.cs — QuizApp.ZonderDatabase › Program.cs › regel 7–44

```
7       static void Main()
8       {
9           QuizData.AddStartQuestions();
10
11          bool running = true;
12          while (running)
13          {
14              Console.WriteLine();
15              Console.WriteLine("=== Quiz-applicatie ===");
16              Console.WriteLine("1. Quiz maken (student)");
17              Console.WriteLine("2. Vragen beheren (docent)");
18              Console.WriteLine("3. Resultaten bekijken (docent)");
19              Console.WriteLine("0. Stoppen");
20              Console.Write("Kies: ");
21              string choice = Console.ReadLine();
22
23              if (choice == "1")
24              {
25                  QuizScreen.Start();
26              }
27              else if (choice == "2")
28              {
29                  QuestionsScreen.Start();
30              }
31              else if (choice == "3")
32              {
33                  ResultsScreen.Start();
34              }
35              else if (choice == "0")
36              {
37                  running = false;
38              }
39              else
40              {
41                  Console.WriteLine("Kies 1, 2, 3 of 0.");
42              }
43          }
44      }
```

Program.cs (regel 7–44)

Wat zeg ik tegen mijn docent

- **Het probleem:** een docent wil weten wat studenten weten. Op papier nakijken kost veel tijd.
- Daaruit kwamen 3 functies (en dus 3 keuzes in het menu): **1)** de student maakt de quiz, **2)** de docent beheert de vragen, **3)** de docent bekijkt de resultaten. Met **0** stop je.
- Het nakijken doet de computer zelf. Dat is de belangrijkste winst voor de docent.

Vragen die de docent kan stellen

V: Hoe ben je van het probleem naar het menu gekomen?

A: Ik heb gekeken wie de app gebruikt (student en docent) en wat ieder wil doen. Elke taak werd een keuze in het menu.

V: Welke functie zou je nog toevoegen?

A: Opslaan in een bestand of database, zodat de resultaten blijven als je de app sluit. En inloggen voor de docent.

V: Wat is een user story bij deze app?

A: Als docent wil ik de resultaten van studenten zien, zodat ik weet wie extra uitleg nodig heeft.

2 Ik kan bepalen welke gegevens mijn applicatie nodig heeft.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → Question.cs + QuizResult.cs (klassen Question, QuizResult, QuizAnswer)

C# Question.cs — QuizApp.ZonderDatabase › Question.cs › regel 1–28

```
1   namespace QuizApp.ZonderDatabase
2   {
3       // Eén vraag uit de quiz.
4       // Is OptionA leeg? Dan is het een open vraag. Anders is het een meerkeuzevraag.
5       public class Question
6       {
7           public string Text { get; set; }
8           public string OptionA { get; set; }
9           public string OptionB { get; set; }
10          public string OptionC { get; set; }
11          public string CorrectAnswer { get; set; }   // "A", "B", "C" of het antwoord van een open vraag
12
13          // Met deze constructor kan ik een vraag in één regel maken.
14          public Question(string text, string optionA, string optionB, string optionC, string correctAnswer)
15          {
16              Text = text;
17              OptionA = optionA;
18              OptionB = optionB;
19              OptionC = optionC;
20              CorrectAnswer = correctAnswer;
21          }
22
23          public bool IsOpenQuestion()
24          {
25              return OptionA == "";
26          }
27      }
28  }
```

Question.cs (regel 1–28)

C# QuizResult.cs — QuizApp.ZonderDatabase › QuizResult.cs › regel 1–26

```
1   using System;
2   using System.Collections.Generic;
3
4   namespace QuizApp.ZonderDatabase
5   {
6       // Eén keer dat een student de quiz heeft gemaakt.
7       public class QuizResult
8       {
9           public string StudentName { get; set; }
10          public int Score { get; set; }
11          public int TotalQuestions { get; set; }
12          public DateTime DateTaken { get; set; }
13
14          // Eén resultaat heeft veel antwoorden (one-to-many).
15          public List<QuizAnswer> Answers { get; set; } = new List<QuizAnswer>();
16      }
17
18      // Het antwoord van een student op één vraag.
19      public class QuizAnswer
20      {
21          public string QuestionText { get; set; }
22          public string GivenAnswer { get; set; }
23          public string CorrectAnswer { get; set; }
24          public bool IsCorrect { get; set; }
25      }
26  }
```

QuizResult.cs (regel 1–26)

Wat zeg ik tegen mijn docent

- Ik heb 3 klassen gemaakt. Een klasse is een soort formulier met vakjes (properties).
- **Question**: de vraag, antwoord A, B, C en het juiste antwoord.
- **QuizResult**: één poging van een student: naam, score, aantal vragen en datum.
- **QuizAnswer**: wat de student bij één vraag heeft gekozen, en of het goed is.
- Een open vraag heeft geen A, B en C. Dat zie je met `IsOpenQuestion()`.

Vragen die de docent kan stellen

V: Waarom bewaar je de vraagtekst in QuizAnswer?

A: Als de docent later de vraag verandert of verwijdert, klopt het oude resultaat nog. Het is een soort foto van dat moment.

V: Wat doet de constructor in Question?

A: Daarmee maak ik een vraag in één regel: `new Question("vraag", "A", "B", "C", "B")`. Zo is QuizData.cs kort.

V: Waarom heeft Question geen Id?

A: Zonder database heb ik geen Id nodig. Ik gebruik de plek in de lijst (de index: 0, 1, 2 ...). In een database heeft elke rij wel een Id nodig.

3 Ik kan een one-to-many-relatie uitleggen en toepassen.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → QuizResult.cs (List<QuizAnswer> Answers) + ResultsScreen.cs → ShowAnswers() (regel 44–57)

C# QuizResult.cs — QuizApp.ZonderDatabase › QuizResult.cs › regel 1–26

```
1   using System;
2   using System.Collections.Generic;
3
4   namespace QuizApp.ZonderDatabase
5   {
6       // Eén keer dat een student de quiz heeft gemaakt.
7       public class QuizResult
8       {
9           public string StudentName { get; set; }
10          public int Score { get; set; }
11          public int TotalQuestions { get; set; }
12          public DateTime DateTaken { get; set; }
13
14          // Eén resultaat heeft veel antwoorden (one-to-many).
15          public List<QuizAnswer> Answers { get; set; } = new List<QuizAnswer>();
16      }
17
18      // Het antwoord van een student op één vraag.
19      public class QuizAnswer
20      {
21          public string QuestionText { get; set; }
22          public string GivenAnswer { get; set; }
23          public string CorrectAnswer { get; set; }
24          public bool IsCorrect { get; set; }
25      }
26  }
```

QuizResult.cs (regel 1–26)

C# ResultsScreen.cs — QuizApp.ZonderDatabase › ResultsScreen.cs › regel 44–57

```
44  private static void ShowAnswers(QuizResult result)
45  {
46      foreach (QuizAnswer answer in result.Answers)
47      {
48          string goodOrWrong = "FOUT";
49          if (answer.IsCorrect)
50          {
51              goodOrWrong = "GOED";
52          }
53
54          Console.WriteLine(goodOrWrong + " " + answer.QuestionText
55              + " gegeven: " + answer.GivenAnswer + " juist: " + answer.CorrectAnswer);
56      }
57  }
```

ResultsScreen.cs (regel 44–57)

Wat zeg ik tegen mijn docent

- **One-to-many** = één ding heeft er veel van een ander ding.
- Eén poging (**QuizResult**) heeft **veel** antwoorden (**QuizAnswer**). Eén antwoord hoort bij maar één poging.
- In mijn code is dat een lijst in de klasse: `public List<QuizAnswer> Answers`.
- Kiest de docent een poging (nummer typen), dan loop ik met `foreach` door `result.Answers` en toon ik alle antwoorden met GOED of FOUT.

Vragen die de docent kan stellen

V: Hoe zou dit in een database gaan?

A: Dan krijg je 2 tabellen. In de tabel QuizAnswers komt een kolom `QuizResultId`. Dat is de **foreign key**: hij wijst naar de poging.

V: Waarom zet je de antwoorden niet als losse velden in QuizResult (Antwoord1, Antwoord2 ...)?

A: Het aantal vragen verandert. Met een lijst maakt het niet uit of er 3 of 37 vragen zijn.

V: Geef een voorbeeld van many-to-many.

A: Student en vak: een student volgt veel vakken, een vak heeft veel studenten.

3. UX & FRONTEND · VAARDIGHEID 1

4 Ik kan een scherm maken dat voor een gebruiker begrijpelijk is.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → QuizScreen.cs → ShowQuestion() (regel 42–52) + Start()

C# QuizScreen.cs — QuizApp.ZonderDatabase › QuizScreen.cs › regel 8–22

```
8       public static void Start()
9       {
10          Console.Write("Jouw naam: ");
11          string name = Console.ReadLine().Trim();
12          if (name == "")
13          {
14              Console.WriteLine("Vul eerst je naam in.");
15              return;
16          }
17
18          if (QuizData.Questions.Count == 0)
19          {
20              Console.WriteLine("Er zijn nog geen vragen. Vraag de docent om vragen te maken.");
21              return;
22          }
```

QuizScreen.cs (regel 8–22)

C# QuizScreen.cs — QuizApp.ZonderDatabase › QuizScreen.cs › regel 42–52

```
42      private static void ShowQuestion(Question question, int index)
43      {
44          Console.WriteLine();
45          Console.WriteLine((index + 1) + " / " + QuizData.Questions.Count + ". " + question.Text);
46          if (!question.IsOpenQuestion())
47          {
48              Console.WriteLine(" A: " + question.OptionA);
49              Console.WriteLine(" B: " + question.OptionB);
50              Console.WriteLine(" C: " + question.OptionC);
51          }
52      }
```

QuizScreen.cs (regel 42–52)

Wat zeg ik tegen mijn docent

- Eerst vraagt de app alleen: "Jouw naam:". Pas daarna komen de vragen.
- De vragen komen één voor één. Voor de vraag staat bijvoorbeeld "3 / 37", zodat de student weet hoe ver hij is.
- Meerkeuze: de app toont A, B en C onder elkaar. Open vraag: geen keuzes, de student typt zelf het antwoord.
- Er staat bij de invoer wat je moet typen: "Jouw antwoord (A, B of C):".

Vragen die de docent kan stellen

V: Waarom eerst alleen de naam?

A: Anders weet ik niet van wie de score is. En de student ziet zo meteen wat de eerste stap is.

V: Waarom `index + 1` bij het nummer?

A: Een lijst begint bij 0. Voor de gebruiker is de eerste vraag vraag 1.

V: Waarom een lege regel (`Console.WriteLine()`) voor elke vraag?

A: Dan staan de vragen niet tegen elkaar aan en is het scherm rustiger om te lezen.

3. UX & FRONTEND · VAARDIGHEID 5

5 Ik kan schermen voorzien van duidelijke labels.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → QuestionsScreen.cs → ReadText() + ReadQuestion() (regel 64–91)

C# QuestionsScreen.cs — QuizApp.ZonderDatabase › QuestionsScreen.cs › regel 64–91

```
64    private static string ReadText(string label)
65    {
66        Console.Write(label + " ");
67        return Console.ReadLine().Trim();
68    }
69
70    private static Question ReadQuestion()
71    {
72        Console.WriteLine("Meerkeuze: vul A, B, C in en als juiste antwoord A, B of C. Open vraag: laat A, B, C
      leeg.");
73        string text = ReadText("Vraag:");
74        string a = ReadText("Antwoord A:");
75        string b = ReadText("Antwoord B:");
76        string c = ReadText("Antwoord C:");
77        string correct = ReadText("Juiste antwoord:");
78
79        string error = CheckInput(text, a, b, c, correct);
80        if (error != "")
81        {
82            Console.WriteLine(error);
83            return null;
84        }
85
86        if (a != "")
87        {
88            correct = correct.ToUpper();
89        }
90        return new Question(text, a, b, c, correct);
91    }
```

QuestionsScreen.cs (regel 64–91)

Wat zeg ik tegen mijn docent

- Voor elke invoer staat een label: "Vraag:", "Antwoord A:", "Juiste antwoord:".
- Bovenaan staat een uitlegregel: hoe maak je een meerkeuzevraag en hoe een open vraag.
- In het hoofdmenu staat achter elke keuze voor wie hij is: "(student)" of "(docent)".

Vragen die de docent kan stellen

V: Waarom zijn labels belangrijk?

A: Zonder label weet de gebruiker niet wat hij moet typen.

V: Waarom `Console.Write` en niet `WriteLine` in ReadText?

A: Met `Write` blijft de cursor achter het label staan. De gebruiker typt dan op dezelfde regel: "Vraag: ...".

V: Waar maak je het label?

A: In `ReadText`. Die toont altijd eerst het label en leest dan de invoer, dus ik kan een label niet vergeten.

3. UX & FRONTEND · VAARDIGHEID 2

6 Ik kan bepalen wanneer een gebruiker feedback nodig heeft.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → QuestionsScreen.cs → AddQuestion() + QuizScreen.cs → FinishQuiz()

C# QuestionsScreen.cs — QuizApp.ZonderDatabase › QuestionsScreen.cs › regel 130–140

```
130    private static void AddQuestion()
131    {
132        Question question = ReadQuestion();
133        if (question == null)
134        {
135            return;
136        }
137
138        QuizData.Questions.Add(question);
139        Console.WriteLine("De vraag is toegevoegd.");
140    }
```

QuestionsScreen.cs (regel 130–140)

C# QuizScreen.cs — QuizApp.ZonderDatabase › QuizScreen.cs › regel 89–111

```
89     private static void FinishQuiz(string name, List<QuizAnswer> answers)
90     {
91         int score = 0;
92         foreach (QuizAnswer answer in answers)
93         {
94             if (answer.IsCorrect)
95             {
96                 score++;
97             }
98         }
99
100        QuizResult result = new QuizResult();
101        result.StudentName = name;
102        result.Score = score;
103        result.TotalQuestions = answers.Count;
104        result.DateTaken = DateTime.Now;
105        result.Answers = answers;
106
107        QuizData.Results.Add(result);
108
109        Console.WriteLine();
110        Console.WriteLine(name + ", je score is " + score + " / " + answers.Count);
111    }
```

QuizScreen.cs (regel 89–111)

Wat zeg ik tegen mijn docent

- Feedback = de app vertelt wat er gebeurd is.
- Ik geef feedback: **1)** als iets gelukt is ("De vraag is toegevoegd."), **2)** als iets fout is ("Vul eerst je naam in."), **3)** vóór iets wat je niet terug kunt draaien (verwijderen).
- Aan het eind van de quiz ziet de student meteen zijn score: "Sam, je score is 30 / 37".

Vragen die de docent kan stellen

V: Waarom een melding na Toevoegen?

A: In een console zie je de lijst niet vanzelf. Zonder melding weet de docent niet of het gelukt is.

V: Kan feedback ook te veel zijn?

A: Ja. Daarom zeg ik na elke vraag niet of het goed of fout is. De student ziet de score aan het eind, en de docent ziet per vraag GOED of FOUT bij de resultaten.

3. UX & FRONTEND · VAARDIGHEID 3

7 Ik kan duidelijke fout- en succesmeldingen maken.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → QuestionsScreen.cs → DeleteQuestion() + CheckInput() (meldingen)

C# QuestionsScreen.cs — QuizApp.ZonderDatabase › QuestionsScreen.cs › regel 160–177

```
160   private static void DeleteQuestion()
161   {
162       int index = AskQuestionNumber();
163       if (index == -1)
164       {
165           return;
166       }
167
168       string sure = ReadText("Deze vraag verwijderen? (j/n):");
169       if (sure.ToLower() != "j")
170       {
171           Console.WriteLine("Niet verwijderd.");
172           return;
173       }
174
175       QuizData.Questions.RemoveAt(index);
176       Console.WriteLine("De vraag is verwijderd.");
177   }
```

QuestionsScreen.cs (regel 160–177)

Wat zeg ik tegen mijn docent

- Een goede foutmelding zegt **wat** er mis is én **wat je moet doen**: "Kies een nummer uit de lijst."
- Bevestiging: "Deze vraag verwijderen? (j/n)". Typ je iets anders dan j, dan stopt de methode (`return`) en zie je "Niet verwijderd."
- Succes: "De vraag is verwijderd.", "De vraag is gewijzigd." en de score aan het eind van de quiz.
- De foutmeldingen van `CheckInput()` staan bij vaardigheid 14.

Vragen die de docent kan stellen

V: Waarom geen technische melding (exception)?

A: De gebruiker is geen programmeur. Hij moet weten wat hij moet doen, niet wat er in de code fout ging.

V: Waarom `sure.ToLower()`?

A: Dan werkt "j" en "J" allebei.

3. UX & FRONTEND · VAARDIGHEID 4

8 Ik kan een logische navigatiestructuur ontwerpen.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → Program.cs → Main() + QuestionsScreen.cs → Start() (menu's met while)

C# Program.cs — QuizApp.ZonderDatabase › Program.cs › regel 7–44

```
7       static void Main()
8       {
9           QuizData.AddStartQuestions();
10
11          bool running = true;
12          while (running)
13          {
14              Console.WriteLine();
15              Console.WriteLine("=== Quiz-applicatie ===");
16              Console.WriteLine("1. Quiz maken (student)");
17              Console.WriteLine("2. Vragen beheren (docent)");
18              Console.WriteLine("3. Resultaten bekijken (docent)");
19              Console.WriteLine("0. Stoppen");
20              Console.Write("Kies: ");
21              string choice = Console.ReadLine();
22
23              if (choice == "1")
24              {
25                  QuizScreen.Start();
26              }
27              else if (choice == "2")
28              {
29                  QuestionsScreen.Start();
30              }
31              else if (choice == "3")
32              {
33                  ResultsScreen.Start();
34              }
35              else if (choice == "0")
36              {
37                  running = false;
38              }
39              else
40              {
41                  Console.WriteLine("Kies 1, 2, 3 of 0.");
42              }
43          }
44      }
```

Program.cs (regel 7–44)

Wat zeg ik tegen mijn docent

- Het hoofdmenu staat in een `while`-lus. Zolang `running` true is, komt het menu steeds terug.
- Elke keuze roept een eigen scherm aan, bijvoorbeeld `QuizScreen.Start()`. Is die methode klaar, dan ben je weer in het menu.
- Vragen beheren heeft een eigen menu met dezelfde opbouw. Met **0** ga je terug. Zo raak je nooit de weg kwijt.
- Menu → Quiz / Vragen / Resultaten → terug naar menu → 0 = stoppen.

Vragen die de docent kan stellen

V: Wat doet `while (running)`?
A: Het herhaalt het menu. Kies je 0, dan wordt `running` false en stopt de lus, en daarmee de app.

V: Wat gebeurt er als je iets anders typt dan 1, 2, 3 of 0?
A: Dan komt de melding "Kies 1, 2, 3 of 0." en zie je het menu opnieuw.

V: Kan een student nu ook het docentmenu openen?

A: Ja, er is nog geen inlog. Volgende stap: inloggen met een rol, en de docent-keuzes alleen aan docenten tonen.

5. PROGRAMMEREN & CODEKWALITEIT · VAARDIGHEID 1

9 Ik kan bestaande code lezen en uitleggen wat deze doet.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → QuizScreen.cs → FinishQuiz() (regel 89–111)

C# QuizScreen.cs — QuizApp.ZonderDatabase › QuizScreen.cs › regel 89–111

```
89      private static void FinishQuiz(string name, List<QuizAnswer> answers)
90      {
91          int score = 0;
92          foreach (QuizAnswer answer in answers)
93          {
94              if (answer.IsCorrect)
95              {
96                  score++;
97              }
98          }
99
100         QuizResult result = new QuizResult();
101         result.StudentName = name;
102         result.Score = score;
103         result.TotalQuestions = answers.Count;
104         result.DateTaken = DateTime.Now;
105         result.Answers = answers;
106
107         QuizData.Results.Add(result);
108
109         Console.WriteLine();
110         Console.WriteLine(name + ", je score is " + score + " / " + answers.Count);
111     }
```

QuizScreen.cs (regel 89–111)

Wat zeg ik tegen mijn docent

- Ik leg deze methode regel voor regel uit:
- `int score = 0;` → ik begin met 0 punten.
- `foreach (QuizAnswer answer in answers)` → ik ga langs alle antwoorden.
- `if (answer.IsCorrect) score++;` → goed? Dan 1 punt erbij.
- Dan maak ik een `QuizResult` en zet ik alles erin: naam, score, aantal, datum en de antwoorden.
- `QuizData.Results.Add(result)` → bewaren in de lijst. Daarna toon ik de score met `Console.WriteLine`.

Vragen die de docent kan stellen

V: Wat doet score++?

A: 1 erbij optellen. Hetzelfde als `score = score + 1`.

V: Verschil tussen foreach en for?

A: foreach gaat langs elk item in een lijst. for gebruikt een teller (i). In `Start()` gebruik ik for, omdat ik het nummer van de vraag nodig heb.

V: Waar komen `name` en `answers` vandaan?

A: Dat zijn parameters. `Start()` geeft ze mee: `FinishQuiz(name, answers)`.

10 Ik kan code opdelen in logische functies/methodes.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → QuizScreen.cs → Start() gebruikt ShowQuestion(), GetGivenAnswer(), CheckAnswer() en FinishQuiz()

C# QuizScreen.cs — QuizApp.ZonderDatabase › QuizScreen.cs › regel 24–40

```
24        List<QuizAnswer> answers = new List<QuizAnswer>();
25        for (int i = 0; i < QuizData.Questions.Count; i++)
26        {
27            Question question = QuizData.Questions[i];
28            ShowQuestion(question, i);
29            string given = GetGivenAnswer(question);
30
31            QuizAnswer answer = new QuizAnswer();
32            answer.QuestionText = question.Text;
33            answer.GivenAnswer = given;
34            answer.CorrectAnswer = question.CorrectAnswer;
35            answer.IsCorrect = CheckAnswer(given, question.CorrectAnswer);
36            answers.Add(answer);
37        }
38
39        FinishQuiz(name, answers);
40    }
```

QuizScreen.cs (regel 24–40)

Wat zeg ik tegen mijn docent

- Elke methode doet één ding:
- `ShowQuestion()` → de vraag (en A, B, C) tonen.
- `GetGivenAnswer()` → het antwoord van de student lezen.
- `CheckAnswer()` → is het goed?
- `FinishQuiz()` → score berekenen en bewaren.
- Zo lees je de lus in `Start()` als een verhaal: 1. vraag tonen, 2. antwoord lezen, 3. bewaren, en aan het eind klaar.

Vragen die de docent kan stellen

V: Waarom is CheckAnswer een aparte methode?

A: Dan staat de regel voor goed/fout op één plek. Wil ik het anders nakijken, dan verander ik maar één methode.

V: Wat geeft GetGivenAnswer terug?

A: Een string: "A", "B", "C" of de getypte tekst. Hij stopt pas als de invoer goed is, dus nooit "".

5. PROGRAMMEREN & CODEKWALITEIT · VAARDIGHEID 4

11 Ik kan herhaalde code refactoren.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → QuestionsScreen.cs → ReadText() en AskQuestionNumber()

C# QuestionsScreen.cs — QuizApp.ZonderDatabase › QuestionsScreen.cs › regel 64–68

```
64    private static string ReadText(string label)
65    {
66        Console.Write(label + " ");
67        return Console.ReadLine().Trim();
68    }
```

QuestionsScreen.cs (regel 64–68)

C# QuestionsScreen.cs — QuizApp.ZonderDatabase › QuestionsScreen.cs › regel 116–128

```
116   private static int AskQuestionNumber()
117   {
118       ShowQuestions();
119       string input = ReadText("Nummer van de vraag:");
120
121       int number;
122       if (!int.TryParse(input, out number) || number < 1 || number > QuizData.Questions.Count)
123       {
124           Console.WriteLine("Kies een nummer uit de lijst.");
125           return -1;
126       }
127       return number - 1;
128   }
```

QuestionsScreen.cs (regel 116–128)

Wat zeg ik tegen mijn docent

- Een label tonen en de invoer lezen zijn 2 regels. Ik doe dat 5 keer voor een vraag, en ook bij andere invoer.
- Daarom heb ik één methode gemaakt: `ReadText("Vraag:")`. Alleen de tekst van het label verschilt.
- `AskQuestionNumber()` gebruik ik bij **Wijzigen** én **Verwijderen**. Het tonen van de lijst en het controleren van het nummer staat dus niet 2 keer in de code.
- Ook `CheckInput()` en `ReadQuestion()` gebruik ik bij Toevoegen én Wijzigen.

Vragen die de docent kan stellen

V: Wat is refactoren?

A: Code netter maken, zonder dat de app iets anders doet.

V: Wat is het voordeel?

A: Minder code. En als ik iets wil veranderen, bijvoorbeeld de melding bij een fout nummer, doe ik dat op één plek.

V: Waarom geeft AskQuestionNumber -1 terug?

A: -1 is nooit een geldige plek in een lijst. Zo weet de methode die hem aanroept: er is iets fout gegaan, stop maar.

12 Ik kan duidelijke namen kiezen voor variabelen, methodes en classes.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → QuizScreen.cs → methodenamen ShowQuestion, GetGivenAnswer, CheckAnswer, FinishQuiz + variabelen

C# QuizScreen.cs — QuizApp.ZonderDatabase › QuizScreen.cs › regel 24–40

```
24        List<QuizAnswer> answers = new List<QuizAnswer>();
25        for (int i = 0; i < QuizData.Questions.Count; i++)
26        {
27            Question question = QuizData.Questions[i];
28            ShowQuestion(question, i);
29            string given = GetGivenAnswer(question);
30
31            QuizAnswer answer = new QuizAnswer();
32            answer.QuestionText = question.Text;
33            answer.GivenAnswer = given;
34            answer.CorrectAnswer = question.CorrectAnswer;
35            answer.IsCorrect = CheckAnswer(given, question.CorrectAnswer);
36            answers.Add(answer);
37        }
38
39        FinishQuiz(name, answers);
40    }
```

QuizScreen.cs (regel 24–40)

Wat zeg ik tegen mijn docent

- Klassen zijn zelfstandige naamwoorden: `Question`, `QuizResult`, `QuizScreen`.
- Een klasse voor een deel van de app eindigt op **Screen**: `QuizScreen`, `QuestionsScreen`, `ResultsScreen`.
- Methodes beginnen met een werkwoord: `ShowQuestion`, `GetGivenAnswer`, `CheckAnswer`, `FinishQuiz`.
- Variabelen zeggen wat erin zit: `given`, `answers`, `score`, `question`.
- C#-regel: PascalCase voor klassen en methodes, camelCase voor variabelen (`isOpenQuestion`).

Vragen die de docent kan stellen

V: PascalCase en camelCase?

A: PascalCase: elk woord met een hoofdletter (ShowQuestion). camelCase: het eerste woord klein (isOpenQuestion).

V: Waarom code in het Engels en schermtekst in het Nederlands?

A: Code in het Engels kan elke programmeur lezen. De gebruiker is Nederlands.

7. TESTEN & KWALITEIT · VAARDIGHEID 2

13 Ik kan happy flows én foutscenario's bedenken.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → QuizScreen.cs → Start() (regel 8–22) + testtabel

C# QuizScreen.cs — QuizApp.ZonderDatabase › QuizScreen.cs › regel 8–22

```
8       public static void Start()
9       {
10          Console.Write("Jouw naam: ");
11          string name = Console.ReadLine().Trim();
12          if (name == "")
13          {
14              Console.WriteLine("Vul eerst je naam in.");
15              return;
16          }
17
18          if (QuizData.Questions.Count == 0)
19          {
20              Console.WriteLine("Er zijn nog geen vragen. Vraag de docent om vragen te maken.");
21              return;
22          }
```

QuizScreen.cs (regel 8–22)

Wat zeg ik tegen mijn docent

- **Happy flow** = de gebruiker doet alles goed. **Foutscenario** = de gebruiker doet iets fout.
- Dit heb ik getest:

#	Wat doet de gebruiker?	Soort	Wat gebeurt er?
1	Naam invullen, alle vragen beantwoorden	happy	Score komt, resultaat staat bij Resultaten
2	Geen naam, meteen Enter	fout	Melding: "Vul eerst je naam in."
3	Docent verwijdert alle vragen, student start de quiz	fout	Melding: "Er zijn nog geen vragen..."
4	Bij een meerkeuzevraag "x" typen	fout	Melding: "Kies A, B of C." en de vraag komt opnieuw
5	In het menu "5" typen	fout	Melding: "Kies 1, 2, 3 of 0."
6	Docent typt bij Verwijderen nummer 99	fout	Melding: "Kies een nummer uit de lijst."

> Tip: probeer dit zelf één keer in de app vóór het gesprek.

Vragen die de docent kan stellen

V: Waarom test je foutscenario's?

A: Gebruikers maken fouten. De app mag dan niet crashen en moet zeggen wat er mis is.

V: Welk probleem is er nog?

A: Sluit je de app, dan zijn de nieuwe vragen en de resultaten weg. Er is geen database of bestand.

14 Ik kan controleren of gebruikersinvoer correct wordt afgehandeld.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → QuestionsScreen.cs → CheckInput() (regel 93–114) + AskQuestionNumber()

C# QuestionsScreen.cs — QuizApp.ZonderDatabase › QuestionsScreen.cs › regel 93–114

```
93    private static string CheckInput(string text, string a, string b, string c, string correct)
94    {
95        correct = correct.ToUpper();
96
97        if (text == "" || correct == "")
98        {
99            return "Vul minstens de vraag en het juiste antwoord in.";
100       }
101
102       bool isOpenQuestion = a == "" && b == "" && c == "";
103       if (!isOpenQuestion && (a == "" || b == "" || c == ""))
104       {
105           return "Vul A, B en C alle drie in, of laat ze alle drie leeg (open vraag).";
106       }
107
108       if (!isOpenQuestion && correct != "A" && correct != "B" && correct != "C")
109       {
110           return "Bij een meerkeuzevraag is het juiste antwoord A, B of C.";
111       }
112
113       return "";
114   }
```

QuestionsScreen.cs (regel 93–114)

C# QuestionsScreen.cs — QuizApp.ZonderDatabase › QuestionsScreen.cs › regel 116–128

```
116   private static int AskQuestionNumber()
117   {
118       ShowQuestions();
119       string input = ReadText("Nummer van de vraag:");
120
121       int number;
122       if (!int.TryParse(input, out number) || number < 1 || number > QuizData.Questions.Count)
123       {
124           Console.WriteLine("Kies een nummer uit de lijst.");
125           return -1;
126       }
127       return number - 1;
128   }
```

QuestionsScreen.cs (regel 116–128)

Wat zeg ik tegen mijn docent

- Voordat een vraag in de lijst komt, controleer ik de invoer met `CheckInput()`.
- 1) Vraag of juiste antwoord leeg → fout.
- 2) Maar 1 of 2 van A, B, C ingevuld → fout (alle 3 of geen).
- 3) Meerkeuze, maar juiste antwoord is niet A, B of C → fout.
- Is alles goed, dan geeft de methode `""` terug en wordt de vraag toegevoegd.
- Bij een nummer gebruik ik `int.TryParse`: typt de docent "abc" of 99, dan crasht de app niet maar komt er een melding.

Vragen die de docent kan stellen

V: Waarom gebruik je CheckInput bij Toevoegen én Wijzigen?
A: Dezelfde regels gelden voor allebei. Nu staan ze op één plek (in `ReadQuestion`).

V: Wat doet ToUpper() hier?
A: "b" wordt "B". Zo mag de docent ook een kleine letter typen.

V: Waarom int.TryParse en niet int.Parse?
A: `int.Parse("abc")` geeft een exception en de app crasht. `TryParse` geeft dan gewoon `false`.

V: Hoe heb je dit getest?
A: Ik heb elk fout geval één keer in de app geprobeerd en gekeken of de juiste melding kwam.

7. TESTEN & KWALITEIT · VAARDIGHEID 5

15 Ik kan edge cases bedenken.

Bewijs (zet dit in je Excel): QuizApp.ZonderDatabase → QuizScreen.cs → GetGivenAnswer() + CheckAnswer() (regel 54–87)

C# QuizScreen.cs — QuizApp.ZonderDatabase › QuizScreen.cs › regel 54–87

```
54    private static string GetGivenAnswer(Question question)
55    {
56        while (true)
57        {
58            if (question.IsOpenQuestion())
59            {
60                Console.Write("Jouw antwoord: ");
61            }
62            else
63            {
64                Console.Write("Jouw antwoord (A, B of C): ");
65            }
66
67            string given = Console.ReadLine().Trim();
68
69            if (given == "")
70            {
71                Console.WriteLine("Typ eerst een antwoord.");
72            }
73            else if (!question.IsOpenQuestion() && given.ToUpper() != "A" && given.ToUpper() != "B" && given.ToUp
      per() != "C")
74            {
75                Console.WriteLine("Kies A, B of C.");
76            }
77            else
78            {
79                return given;
80            }
81        }
82    }
83
84    private static bool CheckAnswer(string given, string correct)
85    {
86        return given.Trim().ToLower() == correct.Trim().ToLower();
87    }
```

QuizScreen.cs (regel 54–87)

Wat zeg ik tegen mijn docent

- Een **edge case** is een randgeval: een rare, maar mogelijke situatie.
- Dit zijn mijn edge cases:

#	Wat doet de gebruiker?	Soort	Wat gebeurt er?
1	Open vraag: student typt "amsterdam " (kleine letter + spatie)	edge	Goed, door `Trim()` en `ToLower()`
2	Meerkeuze: student typt "b" (kleine letter)	edge	Goed, want `ToLower()` maakt beide "b"
3	Student drukt alleen op Enter	edge	Melding, de vraag komt opnieuw en telt niet als fout
4	Laatste vraag beantwoord	edge	De for-lus stopt, daarna `FinishQuiz()`
5	Docent kijkt resultaten, maar er zijn er nog geen	edge	Tekst: "Nog geen resultaten..."
6	Docent typt "b" als juiste antwoord	edge	Wordt "B" (ToUpper)

> Tip: probeer dit zelf één keer in de app vóór het gesprek.

Vragen die de docent kan stellen

V: Welke edge case werkt nog niet?

A: Bij een open vraag telt "7" goed, maar "zeven" fout. De computer vergelijkt alleen de letters.

V: Wat doet `while (true)` in GetGivenAnswer?

A: Hij blijft de vraag stellen tot de invoer goed is. Met `return given;` kom je uit de lus.

V: Wat is het verschil tussen een foutscenario en een edge case?

A: Een foutscenario is iets fout doen. Een edge case is een grensgeval: bijvoorbeeld 0 vragen of de laatste vraag.

ALGEMEEN

Algemene vragen over het hele project

Vragen die de docent kan stellen

V: Waarom een console-app en geen formulieren?

A: Ik wilde eerst de basis goed begrijpen: klassen, lijsten, methodes, if en lussen. Met `Console.WriteLine` en `Console.ReadLine` zie ik precies wat er gebeurt. Formulieren (knoppen, tekstvakken) zijn een volgende stap.

V: Waarom geen database?

A: Om dezelfde reden: eerst de basis. Een database is een extra stap die ik later toevoeg.

V: Wat is het nadeel van geen database?

A: Alles staat in het geheugen. Sluit je de app, dan zijn nieuwe vragen en resultaten weg. Twee computers kunnen de gegevens ook niet delen.

V: Hoe zou je het later opslaan?

A: Stap 1: in een bestand (bijvoorbeeld csv of json). Stap 2: in een database (MySQL). Dan vervang ik `QuizData.Questions.Add(...)` door een INSERT in de database.

V: Wat betekent hard-coded?

A: Gegevens staan vast in de code, bijvoorbeeld de startvragen in `AddStartQuestions()`.

V: Wat is static?

A: Het hoort bij de klasse zelf, niet bij een object. Er is maar één `QuizData.Questions` voor de hele app. Daarom schrijf ik ook `QuizScreen.Start()` zonder `new`.

V: Wat zou je nog verbeteren?

A: 1) Opslaan in een bestand of database. 2) Inloggen voor de docent. 3) Vragen in willekeurige volgorde. 4) Unit tests voor CheckAnswer.

V: Wat heb je zelf gemaakt en wat niet?

A: Beantwoord dit eerlijk. Zeg welke delen je met hulp (docent, internet, AI) hebt gemaakt en wat je daarvan hebt geleerd.